

Docsify Frontend + Pages CMS Backend Integration Documentation

This document explains how to use Docsify as the frontend documentation interface while using Pages CMS (Node.js backend) for authentication and GitHub issue management. The goal is to maintain a static documentation site while delegating login and issue creation features to a centralized backend service.

1. System Overview

The architecture separates the documentation UI from backend services.

Docsify is used to render Markdown documentation as a static site.

Pages CMS backend handles GitHub authentication, issue creation, and API services.

Architecture Flow:

Docsify (Frontend UI)

|

| API Requests

v

Pages CMS Backend (Node.js + Express)

|

| GitHub API

v

GitHub Repository (Issues + Content)

2. Components

Docsify Frontend:

- Static documentation renderer
- Markdown based content
- Sidebar navigation
- Client-side JavaScript integration

Pages CMS Backend:

- GitHub OAuth authentication
- Issue management using GitHub API
- Token generation and validation

- API layer for frontend applications

3. Recommended Folder Structure

```
project-root/  
  cms-backend/  
  server.js  
  routes/  
  controllers/  
  services/  
  .env  
  
  docsify-frontend/  
  index.html  
  README.md  
  _sidebar.md  
  assets/  
  js/
```

4. Authentication Flow

1. User opens Docsify site.
2. User clicks 'Login with GitHub'.
3. Docsify redirects to CMS backend OAuth endpoint.
4. CMS backend authenticates with GitHub.
5. CMS backend returns token or session.
6. Docsify stores token in local storage.
7. Token is used for authenticated API requests.

5. Issue Creation Flow

1. User selects text or opens issue panel in Docsify.
2. Docsify collects issue data (title, description, pageUrl).
3. Docsify sends POST request to CMS backend.

Example API call:

POST /api/issues

Authorization: Bearer

Body:

```
{  
  "title": "Issue title",  
  "description": "Issue description",  
  "pageUrl": "/docs/getting-started"  
}
```

4. CMS backend validates token.
5. CMS backend creates GitHub issue using GitHub API.
6. Response returned to Docsify UI.

6. Required Backend APIs

POST /api/auth/github

Handles GitHub OAuth login.

GET /api/user

Returns authenticated user information.

POST /api/issues

Creates a new GitHub issue.

GET /api/issues?pageUrl=/docs/page

Returns issues associated with a specific documentation page.

7. CORS Configuration

Because Docsify runs as a static frontend, backend must allow cross-origin requests.

Example:

```
app.use(cors({  
  origin: "https://docs.example.com",  
  credentials: true  
}));
```

8. Security Recommendations

- Always use HTTPS.

- Validate GitHub OAuth tokens.
- Use JWT or secure sessions.
- Add rate limiting for issue APIs.
- Sanitize all input fields.

9. Deployment Strategy

Docsify Frontend:

Deploy on static hosting such as:

- GitHub Pages
- Netlify
- Vercel

CMS Backend:

Deploy on server platforms such as:

- Railway
- Render
- VPS
- Vercel server functions

Frontend communicates with backend using REST APIs.

10. Benefits of This Architecture

- Static documentation site with zero backend complexity
- Centralized authentication system
- GitHub issue tracking integrated with documentation
- Clean separation between UI and backend logic
- Easy to scale and maintain